

D&O Policy Intelligence

Relatório técnico do Projeto Final — InsurMinds

Versão: 1.0

Data: 19 de agosto de 2026

Status: MVP funcional validado localmente

Natureza do corpus: documentos sintéticos autorais para demonstração acadêmica

1. Resumo executivo

O D&O Policy Intelligence é um MVP para leitura, estruturação, consulta e comparação de apólices de seguro de responsabilidade civil de Diretores e Administradores — D&O. A aplicação recebe dois documentos em PDF ou imagem, extrai o conteúdo por página usando extração nativa de PDF ou OCR, estrutura campos de interesse com evidências de origem, armazena os resultados em SQLite, permite pesquisa e apresenta diferenças entre as apólices.

A solução foi concebida para atender ao Projeto Final do InsurMinds, que solicita um protótipo capaz de receber documentos, extrair conteúdo, organizar informações, armazená-las, permitir consulta, comparar pelo menos duas apólices e apresentar as principais diferenças com uso de IA Generativa e interface demonstrável [1].

A saída é um apoio à análise documental. Não constitui parecer jurídico, subscrição, decisão de cobertura ou recomendação comercial.

2. Problema e objetivo

Apólices D&O são documentos extensos e jurídicos. A comparação manual de limites, coberturas, exclusões, franquias e condições exige localizar conceitos equivalentes em textos que podem variar em nomenclatura e organização. O objetivo do MVP é reduzir o trabalho mecânico de localização e alinhamento, preservando o trecho original e a página para revisão humana.

O projeto não pretende interpretar toda a legislação securitária nem cobrir todos os tipos de apólice. O foco didático é demonstrar uma arquitetura consistente, modular e explicável em um conjunto delimitado de categorias D&O.

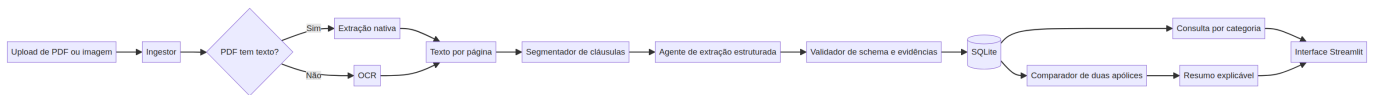
3. Corpus documental

O repositório inclui `data/policies/Apolice_D0_A.pdf` e `data/policies/Apolice_D0_B.pdf`. São documentos sintéticos, autorais e redistribuíveis, criados especificamente para a demonstração. Eles não representam contratos reais nem devem ser utilizados para tomada de decisão securitária.

A estrutura dos documentos foi inspirada em categorias encontradas em condições gerais públicas de D&O, incluindo definições, objetivo, riscos excluídos, limites, franquia, custos de defesa, coberturas básicas, extensões, mudança de controle e período adicional de notificação [2](#). A referência pública serviu como orientação de domínio; o texto entregue no repositório é original e sintético.

Documento	Páginas	Características demonstradas
Apólice A	3	Limite de R10 <i>milhões</i> , <i>franquiade</i> R 50 mil, território sem EUA/Canadá, cobertura C não incluída, período adicional de 24 meses.
Apólice B	3	Limite de R15 <i>milhões</i> , <i>franquiade</i> R 100 mil, território mundial incluindo EUA/Canadá, cobertura C incluída, período adicional de 36 meses.

4. Arquitetura da solução



O fluxo é composto por componentes com responsabilidades separadas:

Componente	Responsabilidade	Implementação
Recepção	Validar extensão, existência e quantidade de documentos	<code>seguramente_final/ingest.py</code>
Ingestor	Ler PDF nativo ou imagem e produzir texto por página	<code>load_document</code> , <code>extract_pdf_pages</code> , <code>extract_image_pages</code>
OCR	Reconhecer páginas sem texto extraível	<code>pytesseract</code> com <code>pdf2image</code>
Schema	Definir campos comparáveis e categorias D&O	<code>seguramente_final/schema.py</code>
Agente de extração	Mapear texto para fatos estruturados	<code>RuleBasedExtractor</code> ou <code>OpenAICompatibleExtractor</code>
Validador	Exigir status, valor, página e evidência	validação local do objeto <code>PolicyFact</code>
Persistência	Guardar documentos, páginas, fatos e comparações	<code>seguramente_final/storage.py</code> com SQLite
Consulta	Filtrar categoria, campo, valor ou evidência	interface Streamlit
Comparador	Alinhar duas apólices pelo schema e classificar diferenças	<code>seguramente_final/compare.py</code>
Apresentação	Exibir resumo, tabela lado a lado e evidências	<code>app.py</code>

5. Agentes e responsabilidades

A solução organiza o processamento em componentes especializados, conforme a orientação do edital [1].

Agente	Entrada	Ação	Saída	Limite
Recepção de Documentos	PDF ou imagem	Validar arquivo e calcular identificador SHA-256	Documento registrado	Não interpreta cláusulas
Extração/OCR	Arquivo e páginas	Extrair texto nativo ou aplicar OCR	Texto por página e método	Não cria conteúdo ausente
Extração Estruturada	Texto e schema	Identificar campos, valores, status e evidências	PolicyFact	Não infere cobertura sem trecho
Validação e Normalização	Fatos retornados	Normalizar espaços, moeda e status; localizar evidência	Fatos verificáveis	Não substitui revisão humana
Armazenamento	Documentos e fatos	Persistir em tabelas relacionais	SQLite	Não mantém histórico externo
Comparação	Fatos de A e B	Alinhar campos e classificar igualdade/diferença/ausência	ComparisonRow	Não produz decisão jurídica
Relatório	Linhas comparativas	Gerar resumo com categorias e ressalva	Resumo explicável	Não recomenda contratação

6. Schema de dados D&O

O schema delimita 19 campos comparáveis. Cada campo possui categoria, label, valor textual, status, confiança, página, evidência e valor normalizado.

Categoria	Campos
Identificação	Seguradora; vigência; âmbito geográfico
Limites	Limite máximo de responsabilidade; limite agregado
Retenções	Franquia / retenção
Coberturas	Cobertura A; Cobertura B; Cobertura C; custos de defesa
Extensões	Custos de investigação; novas subsidiárias; período adicional de notificação
Exclusões	Fraude ou dolo; danos corporais ou materiais; poluição
Condições	Notificação de reclamações; mudança de controle; acordos e consentimento

Os estados possíveis são `present`, `not_found`, `ambiguous` e `not_applicable`. A aplicação não converte ausência de localização em exclusão de cobertura. Essa distinção é essencial para evitar conclusões jurídicas indevidas.

Exemplo de fato estruturado:

```
{
  "category": "limites",
  "label": "Limite máximo de responsabilidade",
  "value": "R$ 10.000.000,00",
  "status": "present",
  "confidence": 0.98,
  "page": 2,
  "evidence": "LIMITE MÁXIMO DE RESPONSABILIDADE: R$ 10.000.000,00"
}
```

7. Extração de PDF e OCR

Para PDF pesquisável, o sistema utiliza `pypdf` e preserva o texto por página. Quando uma página retorna menos de 20 caracteres, o pipeline tenta convertê-la em imagem com `pdf2image` e aplica `pytesseract` nos idiomas português e inglês. Cada página registra o método utilizado: `pdf-text`, `ocr` ou estado vazio.

Para imagens PNG, JPG, JPEG, TIFF e similares, o sistema aplica OCR diretamente. Arquivos com extensão não suportada, vazios ou inexistentes são rejeitados com erro controlado.

8. Uso de IA Generativa

O provider `OpenAICompatibleExtractor` utiliza endpoint compatível com Chat Completions e solicita resposta JSON. O modelo recebe apenas o texto das páginas e a lista de campos autorizados. A instrução exige que não sejam inventados valores, que campos não localizados recebam `not_found` e que o resultado contenha evidência e página quando disponível.

O resumo comparativo também pode ser produzido pelo modelo a partir das linhas já estruturadas. A comparação principal, entretanto, é determinística e executada sobre fatos validados. Isso evita que o resumo livre seja a única fonte da conclusão.

O provider `RuleBasedExtractor` existe para execução offline, testes e demonstração sem credencial. Se a resposta JSON da IA vier incompleta, o MVP aplica um fallback determinístico local para evitar lacunas artificiais, mantendo o uso do provider de IA registrado no resultado. Essa decisão é documentada e torna a demonstração reproduzível sem mascarar falhas do modelo.

9. Comparação e consulta

As duas apólices são alinhadas pelo par `categoria + label`. Cada linha de comparação recebe um dos estados: `igual`, `diferente`, `ausente_em_a`, `ausente_em_b`, `ambigua` ou `ambos_ausentes`. A interface mostra o valor de cada apólice, o status, a descrição da diferença e as páginas de origem.

A consulta textual filtra categoria, campo, valor e evidência. A área de evidências permite expandir cada item e verificar lado a lado o trecho das duas apólices.

10. Persistência

O SQLite contém as tabelas `documents`, `pages`, `facts` e `comparisons`. A tabela de documentos guarda identificador, nome, caminho e origem. A tabela de páginas mantém texto e método de extração. A tabela de fatos guarda status, confiança, página, evidência e normalização. A tabela de comparações registra os valores, diferenças, status e evidências de A e B.

11. Interface de demonstração

A interface Streamlit possui duas modalidades. No modo corpus sintético, utiliza as duas apólices incluídas no projeto. No modo upload, aceita dois arquivos em PDF ou imagem e salva cópias temporárias para a execução.

A demonstração visual apresenta os documentos recebidos, contagem de páginas e métodos de extração; resumo comparativo; consulta por campo; tabela lado a lado; evidências por página; e limitações. O sistema exige duas apólices para executar a comparação.

12. Validação executada

Validação	Resultado
Testes automatizados	6 aprovados
Documentos processados no demo offline	2
Páginas processadas	6
Campos comparados	19
Diferenças/lacunas identificadas	11
Provider offline	rule-based-offline
Provider com IA	openai-compatible / gpt-5-mini
Smoke test Streamlit	Aplicação iniciada e respondeu localmente

As evidências estão em `evidence/demo_analysis_offline.json` e `evidence/demo_analysis_openai.json`. Os arquivos não contêm chaves de API.

13. Instalação e execução

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
pytest -q
streamlit run app.py
```

Para executar a demonstração offline:

```
python3 scripts/run_demo.py --provider offline --output evidence/demo_analysis_offlin
```

Para executar a integração com IA:

```
python3 scripts/run_demo.py --provider openai --output evidence/demo_analysis_openai.
```

A variável `OPENAI_API_KEY` deve estar no `.env`. O modelo pode ser ajustado por `SEGURAMENTE_LLM_MODEL`; o ambiente validado utilizou `gpt-5-mini`.

14. Limitações conhecidas

O corpus é sintético e não representa apólices comerciais. O OCR pode produzir erros em tabelas, colunas, números e caracteres jurídicos. O schema cobre um conjunto delimitado de campos e não pretende interpretar todos os endossos, condições particulares ou conflitos de cláusulas. A comparação textual e normalizada é apoio à revisão; não substitui especialista em seguros ou advogado.

O projeto não implementa autenticação, alta disponibilidade, controle de acesso multiusuário, integração com sistemas de seguradoras ou assinatura digital. Essas limitações são coerentes com a orientação do edital, que prioriza um MVP compreensível e demonstrável em vez de produto comercial completo [1].

15. Evoluções futuras

As evoluções prioritárias são: incorporar recuperação contextual por cláusula, adicionar classificação de confiança baseada em validação cruzada, permitir revisão e correção humana dos fatos, suportar múltiplas versões de uma mesma apólice, exportar relatório comparativo em PDF e integrar um corpus público com licenças verificadas.

16. Rastreabilidade ao edital

Requisito	Evidência no projeto
Leitura de PDF ou imagem	<code>ingest.py</code> , <code>pypdf</code> , <code>pytesseract</code> e upload Streamlit
Extração automática	<code>RuleBasedExtractor</code> e <code>OpenAICompatibleExtractor</code>
Estruturação dos dados	<code>schema.py</code> , <code>PolicyFact</code> e JSON de evidência
Armazenamento estruturado	SQLite em <code>storage.py</code>
Comparação de pelo menos duas apólices	<code>compare.py</code> , demo e tabela de 19 campos
Principais diferenças ao usuário	DataFrame comparativo e evidências por página
Uso de IA Generativa	Provider OpenAI-compatible e resumo comparativo
Interface demonstrável	<code>app.py</code> com Streamlit
Relatório técnico	Este documento em PDF e Markdown
Pitch Deck e vídeo	<code>Projeto_Final_Artefatos/InsurMinds_Projeto_Final.pptx</code> e <code>.mp4</code>
Repositório e ZIP	README, LICENSE, pacote e repositório público

Referências

[1]: `Desafios(1).pdf` , Instituto de Inteligência Artificial Aplicada — I2A2, páginas 20–25, edital fornecido para o Projeto Final.